arXiv:2011.01387v2 [cs.RO] 11 Mar 2021

通过周期性奖励组合实现所有常见双足步态的仿真到现实学习

Jonah Siekmann*, Yesh Godse*, Alan Fern, Jonathan Hurst 俄勒冈州立大学协作机器人与智能系统研究所{siekmanj, godsey, afern, jhurst}@oregonstate.edu

摘要—我们研究了通过仿真到现实强化学习（***RL***）在真实机器人上实现完整双足运动范围的问题。学习腿式运动的一个关键挑战是如何通过奖励函数描述不同的步态，使其对设计者直观，并且足够具体，以便在不同的初始随机种子或超参数下可靠地学习该步态。一种常见的方法是使用参考运动（例如关节位置轨迹）来指导学习。然而，找到高质量的参考运动可能很困难，而且这些轨迹本身会严格限制学习到的运动空间。另一个极端是，无参考的奖励函数往往定义不足（例如向前移动），导致策略行为出现巨大差异，或者通过反复试错进行大量奖励塑形，使其仅适用于特定步态。在这项工作中，我们提出了一种基于在基本力和速度上组合简单概率周期代价的奖励规范框架。我们实例化该框架，定义了一个参数化奖励函数，对所有常见双足步态——站立、行走、双脚跳、跑步和蹦跳——具有直观的设置。利用该函数，我们成功地将学习到的步态仿真到现实迁移到双足机器人***Cassie***上，并展示了一种能够在所有两拍步态之间转换的通用策略。

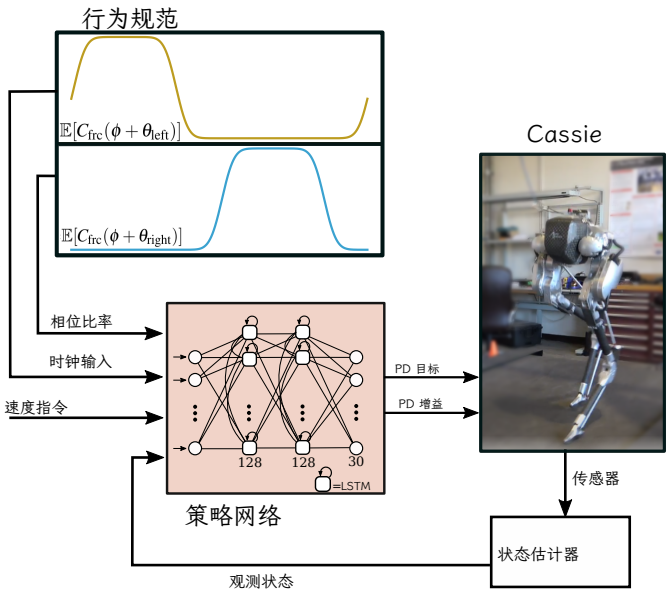


图1. 在这项工作中，我们提出了一种奖励设计框架，使得在硬件上学习能够站立、行走、跑步、疾驰、单脚跳和蹦跳的策略变得容易。我们基于多个步态参数来调节奖励函数，并将这些参数提供给LSTM策略，该策略向机器人输出PD关节位置目标和PD增益。

一、引言

使用强化学习（RL）让真实机器人学会自然界中所有常见双足步态仍是一个尚未解决的问题。通过RL学习特定运动步态的一个关键挑战是如何通过奖励函数传达步态行为。一般来说，特定步态可以看作一个动态过程，它具有特征性的周期结构，同时也能够灵活适应中等程度的环境扰动。这为设计步态奖励函数提出了两点考虑。首先，奖励必须足够具体，以便在优化时产生所需的步态特征。其次，考虑到在……的背景下，步态的具体细节存在不确定性，

在特定的地形和动态条件下，奖励不应设置过多限制。

使用参考轨迹来指定步态特定奖励的常见做法，例如 [1]–[5]，，部分解决了上述第一个考虑因素，但在很大程度上忽略了第二个。特别是，参考轨迹仅捕捉了在不同条件下实现步态特征所需变化的一小部分。因此，尝试遵循这样的轨迹可能会妨碍学习一种更稳健和/或更高效的步态特征，更不用说推导出针对特定期望步态的可行参考轨迹本身就可能非常具有挑战性。

无参考的移动奖励函数指定方法往往高度欠约束，例如 OpenAI Gym [6]移动基准测试中所用的方法。以此为起点，要实现特定的步态特征，需要基于观察到的强化学习性能反复进行启发式奖励函数调整，直到达到期望行为。这种方法即使可行也往往冗长乏味，且作为通用框架并不可靠。其他无参考方法则将奖励围绕特定类型的移动行为 [7] 进行构建，但难以扩展到其他行为。

我们工作的第一个贡献是提出了一个原则性框架，用于设计能够自然捕捉所有周期性双足行走步态的奖励函数。我们的动机源于这样一个事实：所有常见的双足步态都可以由每只脚[8]的周期性摆动相（脚在空中摆动）和支撑相（脚着地）来定义。摆动相与支撑相之间的一个根本区别

* 表示同等贡献，顺序由掷硬币决定。

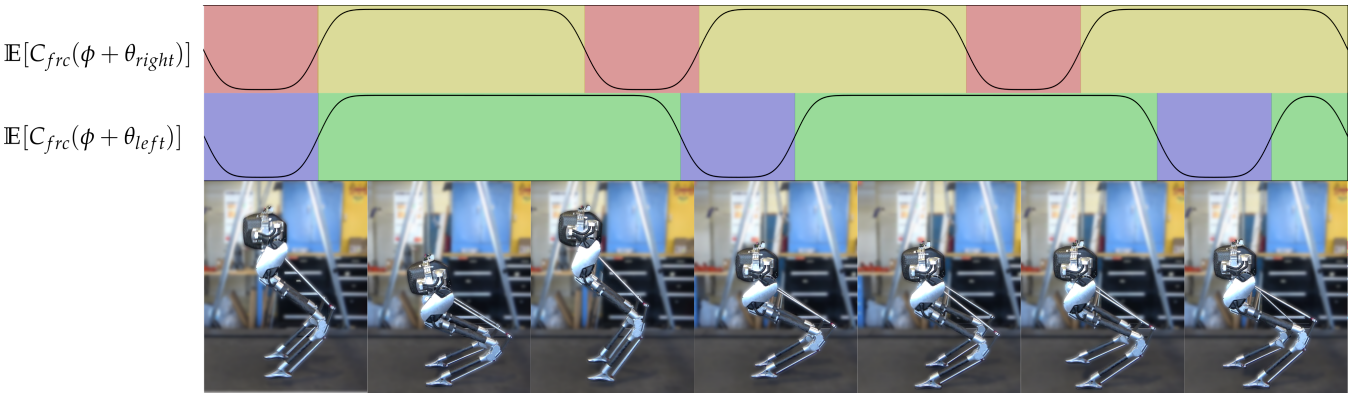


图2：一系列图像展示了神经网络策略控制 Cassie 并连续地从跳跃过渡到疾驰再到行走。我们提出了一种简单的奖励设计范式，利用概率区间在特定时间应用代价函数，使策略能够学习自然界动物表现出的所有常见双足步态。

阶段是特定脚上脚力与脚速度的互补性存在和缺失。基于这一观察，我们可以通过利用脚力和脚速度的大小来设计原则性奖励函数，使得在摆动相期间，脚力受到惩罚而速度被允许，从而促使策略学会抬脚。因此，我们的框架将步态描述为周期性阶段的序列，每个阶段奖励或惩罚物理系统的某一特定测量量。我们的假设是，这一框架将允许更自然地指定奖励函数，足以充分约束强化学习去学习期望的步态特征，同时允许灵活适应特定的环境干扰。

我们的第二个贡献是展示该框架在仿真到现实强化学习中的应用，涵盖行走、跑步、疾驰、蹦跳和跳跃等所有常见双足步态，且无需使用动作捕捉数据集或参考轨迹。我们在仿真中为每种行为训练策略，并在硬件上成功演示。此外，通过将框架的步态参数作为命令输入提供给策略，我们能够学习一种多步态策略，使其可以跳跃、疾驰、跑步和行走。

II. 背景

合成运动行为的实践已在多个领域得到研究。在角色动画中，基于运动学的方法（即使用动作捕捉数据的方法）通常用于合成运动[9] [10] [11]，，一些较新的方法则利用深度学习 [1] 训练神经网络，以解决诸如自适应不同地面几何形状 [12] [13] 或将多种动作融合为一种逼真身体运动 [2] 等问题。基于物理的角色动画，即通过施加力矩控制角色姿态并使用物理引擎模拟运动，近年来也日益受到重视 [14]，，但与基于运动学的方法相比，它要有效处理复杂运动 [15] 的难度明显更大。

使用强化学习来合成运动的方法与前述方法具有很大的相似性。角色动画中使用的方法。许多近期工作使用轨迹匹配奖励函数来训练策略，使其模仿某个参考轨迹（类似于动作捕捉），同时让策略受到模拟物理的约束，从而生成既能逼真地模仿参考轨迹又能真实行动的策略 [16] [5] [3] [17] [4]。强化学习也被用于无需参考轨迹的运动合成，但这类方法往往不重视行为的主观质量，所产生的行为虽然能最大化某些目标，但相应的策略在现实世界中常常效率低下、不可行或不安全，而且通常也不美观 [6] [18]。确实存在不使用参考轨迹而优先考虑物理真实行为的方法 [19] [20] [7]，，但它们的奖励函数只针对单一行为，难以推广到其他行为。

III. 利用周期奖励组合学习双足步态

A. 强化学习框架

我们在强化学习（RL） [21]，框架下表述我们的问题，并假定读者对此有基本了解。世界被建模为一个离散时间的马尔可夫决策过程（MDP），具有连续状态空间S、连续动作空间A、转移函数T(s,a，s′)和奖励函数R(s,t)。其中，T(s,a,s′)给出在状态s下采取动作a后，下一个状态s′ 的概率密度；R(s,t)给出在时间步t处于状态s时获得的非平稳奖励。

控制策略是一个从状态到动作的可能随机的映射 $\pi(a \mid s)$，它决定行为。给定一个策略，期望的T-步折扣回报由 $J(\pi) = \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t R(S_t,t)\right]$给出，其中 $\gamma \in [0,1]$ 是折扣因子，S_t 是一个随机变量，表示在遵循策略 π 并在转移动力学T下时间t的状态。强化学习的目标是基于世界中的试错训练经验，学习使J(π)最大化的策略 π 。在本工作中，我们遵循一种从仿真到现实的强化学习范式，其中

training is done in simulation to identify a policy, which is then used in the real-world.

B. Periodic Reward Composition

Since our framework is targeted toward periodic behaviors, we index time via a *cycle time* ϕ variable, which repeatedly cycles over a normalized time period of $[0,1]$ at discrete time steps rather than increasing monotonically. Accordingly, the non-stationary reward function $R(s,\phi)$ is periodic and defined in terms of ϕ rather than absolute time.

As motivated in the introduction, our framework specifies rewards in terms of compositions of rewards on periodic intervals. We define the reward as a biased sum of *n reward components* $R_i(s,\phi)$, where each component $R_i(s,\phi)$ captures a desired characteristic of the gait during a particular phase.

$$R(s,\phi) = \beta + \sum_i R_i(s,\phi)$$

Each reward component $R_i(s,\phi)$ is a product of a *phase coefficient* c_i, a *phase indicator* $I_i(\phi)$, and a real-valued *phase reward measurement* $q_i(s)$ (e.g. norm of a foot force).

$$R_i(s,\phi) = c_i \cdot I_i(\phi) \cdot q_i(s)$$

The phase coefficient c_i is a scalar whose sign determines the effect that the phase measurement $q_i(s)$ has on the total reward during cycle times when the reward component is active. The phase indicator function $I_i(\phi)$ is a binary-valued random variable denoting whether the target phase is active or not at cycle time ϕ. In this work, the distribution of each I_i is described by random variables A_i and B_i representing the start and end times of the period respectively. Since A_i and B_i represent intervals on a cycle, we use Von Mises distributions (approximations of the wrapped Normal distribution) described by the parameter tuple (a_i, b_i, κ), which gives the means a_i and b_i and shared variance parameter κ. The distribution of the binary phase indicator $I_i(\phi)$ is then simply:

$$P(I_i(\phi) = x) = \begin{cases} P(A_i < \phi < B_i) & \text{if } x = 1 \\ 1 - P(A_i < \phi < B_i) & \text{if } x = 0 \end{cases} \quad (1)$$

$$\text{where } P(A_i < \phi < B_i) = P(A_i < \phi)(1 - P(B_i < \phi))$$
$$A_i \sim \Phi(2\pi a_i, \kappa) \text{ and } B_i \sim \Phi(2\pi b_i, \kappa)$$
$$\Phi \text{ is the Von Mises distribution}$$

Note that this formulation allows for uncertainty about the exact start and end of each phase to be captured via the variance parameter κ of the Von Mises distributions. This has a smoothing effect on the reward function at phase boundaries, which we have found to usefully encourage more stable and consistent learning. While we could directly use this probabilistic reward function $R(s,\phi)$ for RL training by sampling from the reward distribution at each step, we instead apply RL to the deterministic expectation of $R(s,\phi)$.

$$\mathbb{E}[R(s,\phi)] = \sum_i^n c_i \cdot \mathbb{E}[I_i(\phi)] \cdot q_i(s) + \beta$$

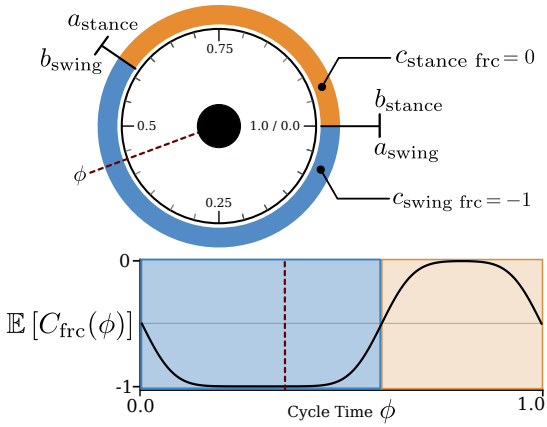


Fig. 3. The circular intervals of two phases, swing and stance, are shown on a polar plot. The expected sum of the phase indicators and phase coefficients for foot force $C_{\text{frc}}(\phi)$ is shown below.

Due to the linearity of expectations, for any policy π the expected cumulative return $J(\pi)$ is the same regardless of whether we use stochastic rewards or their expectations.

C. Describing Bipedal Gaits

For simplicity, we begin by describing repeatedly lifting and placing a single foot, or equivalently, cycling between swing and stance phases with our framework. As our phase reward measurements, we select the norm of foot force $q_{\text{frc}}(s)$ and the norm of foot velocity $q_{\text{spd}}(s)$. During the swing phase we want to penalize foot forces and ignore foot velocities, so we choose $c_{\text{swing frc}} = -1$ and $c_{\text{swing spd}} = 0$. Similarly, we choose $c_{\text{stance spd}} = -1$ and $c_{\text{stance frc}} = 0$ to penalize foot velocities and ignore foot forces during the stance phase. We constrain the swing and stance phases to follow immediately after one another and together last the entire cycle time by defining a ratio $r \in (0,1)$ and setting the intervals for both phases such that the swing phase lasts length r, while the stance phase lasts length $1-r$ and starts directly afterwards. Finally, by choosing a common scale κ, we can define the indicator functions $I_{\text{frc}}(\phi)$ and $I_{\text{spd}}(\phi)$ by using Equation 1.

For convenience, we define $C_{\text{frc}}(\phi)$ and $C_{\text{spd}}(\phi)$ as the expected sum of the product of all the phase indicators and phase coefficients for the swing and stance phases:

$$\begin{aligned} \mathbb{E}[C_{\text{frc}}(\phi)] = \; & c_{\text{swing frc}} \cdot \mathbb{E}[I_{\text{swing frc}}(\phi)] \\ & + c_{\text{stance frc}} \cdot \mathbb{E}[I_{\text{stance frc}}(\phi)] \\ \mathbb{E}[C_{\text{spd}}(\phi)] = \; & c_{\text{swing spd}} \cdot \mathbb{E}[I_{\text{swing spd}}(\phi)] \\ & + c_{\text{stance spd}} \cdot \mathbb{E}[I_{\text{stance spd}}(\phi)] \end{aligned}$$

Refer to Fig.3 for a visual explanation of the phase start and end times, ϕ, and the expected value of C_{frc}. For more complicated behaviors, we find that visualizing $C_{\text{frc}}(\phi)$ can be useful for understanding how the reward function changes over the cycle to guide the learning of a particular behavior.

Putting the force and speed components together, the expected overall reward for repeatedly lifting and placing a single foot is

$$\begin{aligned} \mathbb{E}[R_{\text{unipedal}}(s,\phi)] = \; & \mathbb{E}[C_{\text{frc}}(\phi)] \cdot q_{\text{frc}}(s) \\ & + \mathbb{E}[C_{\text{spd}}(\phi)] \cdot q_{\text{spd}}(s) \end{aligned}$$

Bipedal gaits are behaviors where the left and right feet both follow the same sequence of phases described above, but offset relative to each other in phase time. For instance, in a walking behavior the timings of the swing and stance phases are shifted apart by half of the period length (one leg in swing, the other in stance), while a hopping behavior is one where both feet synchronously enter the swing and stance phases. To expand the simple behavior of lifting and placing a single foot into the full spectrum of bipedal gaits, we need only introduce two *cycle offset* parameters $\theta_{\text{left}}, \theta_{\text{right}}$ which define the exact timing shift between the identical sequence of behavioral phases for the left and right feet, and differentiate between the norms of left and right foot forces, $q_{\text{left frc}}(s)$, $q_{\text{right frc}}(s)$ and norms of left and right foot velocities $q_{\text{left spd}}(s)$, $q_{\text{right spd}}(s)$. The expected overall reward for a bipedal behavior is

$$\begin{aligned}\mathbb{E}[R_{\text{bipedal}}(s,\phi)] = \quad & \mathbb{E}[C_{\text{frc}}(\phi+\theta_{\text{left}})]\cdot q_{\text{left frc}}(s) \\ & +\mathbb{E}[C_{\text{frc}}(\phi+\theta_{\text{right}})]\cdot q_{\text{right frc}}(s) \\ & +\mathbb{E}[C_{\text{spd}}(\phi+\theta_{\text{left}})]\cdot q_{\text{left spd}}(s) \\ & +\mathbb{E}[C_{\text{spd}}(\phi+\theta_{\text{right}})]\cdot q_{\text{right spd}}(s)\end{aligned} \tag{2}$$

Introducing $\theta_{\text{left}}, \theta_{\text{right}}$ for two phase behaviors allows us to define reward functions for walking, galloping, and hopping. Additionally, by using four phases rather than two in each component, we can derive a skipping reward function, as shown in Fig. 4.

IV. Method

Network Architecture and Action Space: For all policies, we use a Long Short-Term Memory neural network [22] with two recurrent hidden layers of size 128 each, and a simple linear output projection of size 30, corresponding to 10 desired joint positions, and 10 sets of PD gains, as seen in Fig. 1. The desired joint positions are added to a set of constant offsets corresponding to a neutral standing position, such that an output vector of zeroes results in a standing pose. Similarly, the P gains and D gains are also summed with a fixed 'neutral' set of gains. The policy is evaluated at 40Hz, and the robot's PD controllers are run at 2000Hz. A similar system is used in [3] and [4], though without PD gain deltas.

State Space: To prevent the reinforcement learning environment from becoming a non-stationary Markov decision process, some information about the periodic reward function must be present in the state provided to the policy. Specifically, the policy must receive some sort of encoding of the current cycle time ϕ, an encoding of the cycle offset parameters, $\theta_{\text{left}}, \theta_{\text{right}}$, and information about the start and end times of the phases of the phase reward components.

In order to encode the current cycle time ϕ and the cycle offset parameters $\theta_{\text{left}}, \theta_{\text{right}}$, we condition the policies on two clock inputs:

$$p = \left\{ \sin\left(\frac{2\pi(\phi+\theta_{\text{left}})}{L}\right), \sin\left(\frac{2\pi(\phi+\theta_{\text{right}})}{L}\right) \right\}$$

where L is the number of discrete timesteps in the entire cycle. To encode information about the start and end times of the phases into the state, we derive a vector of *ratios* from the sequence of phase timings; each ratio represents the proportion out of the total period that a phase occupies. For instance, a phase j with start time $a_{i,j=0.3}$ and end time $b_{i,j}=0.7$ should occupy 40% of the total period time (plus or minus some uncertainty); thus, we can derive a ratio $r_j = 0.4$. Each phase's ratio is calculated and provided to the policy.

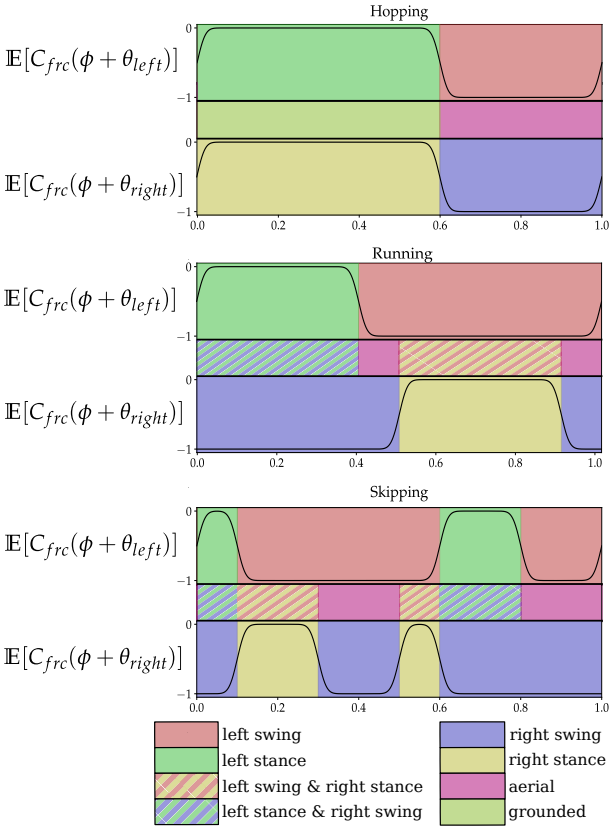


Fig. 4. Plot of the expected value of the force phase coefficient for each foot for some example bipedal gaits. In hopping there is a $|\theta_{\text{left}} - \theta_{\text{right}}| \approx 0$ shift between the left and right feet. For walking and running, $|\theta_{\text{left}} - \theta_{\text{right}}| \approx 0.5$, with transitional shifts resulting in galloping. Four phases are necessary for describing skipping, as well as a $|\theta_{\text{left}} - \theta_{\text{right}}| \approx 0.5$ shift between the left and right feet. The shaded region between the phase coefficient plots show the hybrid phases from combining the left and right foot behaviors together.

Thus, the policy's input consists of:

$$X_t = \begin{cases} \hat{q},\ \hat{\dot{q}} & \text{robot state} \\ \dot{x}_{\text{desired}}, \dot{y}_{\text{desired}} & \text{desired velocity} \\ r, p & \text{phase ratios and clock inputs} \end{cases}$$

Where $\hat{q}, \hat{\dot{q}}$ are estimates of the pelvis orientation, rotational velocity, joint positions and joint velocities. $\dot{x}_{\text{desired}}$ and $\dot{y}_{\text{desired}}$ are speed commands randomized during training, and manually controlled by the user during evaluation.

Reward Formulation: We use the set of reward components from R_{bipedal} defined in Equation 2 to learn single-gait policies. In addition, we also use a variety of auxiliary cost components to further constrain the path of viable exploration towards stable locomotion and facilitate successful sim-to-real transfer as well as command the policy to match a desired orientation, forward speed, and sidespeed. These rewards do not need to vary as a function of time, and can be

seen as a special case of the reward component framework, wherein the random variable c_i has only one phase, and thus one possible value (in this case, -1, to show that we wish to penalize these quantities q_i for the entire period).

$$\begin{aligned} R_{\text{cmd}}(s) = &\quad (-1)\cdot q_{\dot{x}}(s) \\ &+ \quad (-1)\cdot q_{\dot{y}}(s) \\ &+ \quad (-1)\cdot q_{\text{orientation}}(s) \\ R_{\text{smooth}}(s) = &\quad (-1)\cdot q_{\text{action diff}}(s) \\ &+ \quad (-1)\cdot q_{\text{torque}}(s) \\ &+ \quad (-1)\cdot q_{\text{pelvis acc}}(s) \end{aligned}$$

$$\mathbb{E}[R(s,\phi)] = \mathbb{E}[R_{\text{bipedal}}(s,\phi)] + R_{\text{smooth}}(s)] + R_{\text{cmd}}(s) + \beta \quad (3)$$

Where $q_{\dot{x}}$ and $q_{\dot{y}}$ are measures of error between the commanded velocity and the actual pelvis velocity, and $q_{\text{orientation}}$ is the negative exponent of quaternion difference between the pelvis orientation and an orientation which faces straight, which can be used to change the heading of the robot. $q_{\text{pelvis acc}}$ is a cost for aggressive pelvis motion, which helps reduce noise in the state estimator, while $q_{\text{action diff}}$ is a cost for aggressive actions and q_{torque} is an overall joint torque cost to encourage efficient motions. For general policies which can learn and transition between a continuum of bipedal gaits, we specify additional reward terms which are discussed in Section V.

Dynamics Randomization: In order to facilitate successful sim-to-real transfer, we use dynamics randomization [23] [24] to expose the policies to a wide variety of possible real-world dynamics. Specifically, we randomize the execution rate of the policy, the mass and damping of the joints, the friction of the ground, the slope of the ground, and joint position encoder noise. Details on the ranges and distributions used to randomize these parameters can be found in Table I. These parameters are randomized at the beginning of every rollout during training and remain constant throughout each rollout.

Parameter	Unit	Range
Joint damping	Nms/rad	$[0.3, 4.0] \times$ default values
Joint mass	kg	$[0.5, 1.5] \times$ default values
Ground Friction	–	$[0.35, 1.1]$
Ground Slope	rad	$[-0.03, 0.03]$
Joint Encoder Offset	rad	$[-0.05, 0.05]$

TABLE I. The ranges for randomization of several dynamics parameters during training. We use a uniform distribution over the given ranges for all listed parameters.

Proximal Policy Optimization: To train our policies, we use a common model-free reinforcement learning algorithm known as Proximal Policy Optimization (PPO) [25]. More specifically, we use the recurrent adaptation described in [4], which samples batches of trajectories from a replay buffer, rather than batches of individual timesteps. In our case, we use a recurrent critic with exactly the same dimensions as the actor with the exception of the final linear output vector, which in the case of the critic is a scalar. We use a fixed exploration action noise, with standard deviation e^{-1}. In addition, we incorporate a mirror loss term [26] into our policy gradient algorithm with the aim of achieving more symmetric locomotion behavior.

V. Experimental Results

Training Details. Policies were trained using PPO with a batch size of 32 trajectories of up to 300 timesteps each, a learning rate of 0.0001 for both the actor and critic, a replay buffer of 50,000 samples, and 4 epochs per iteration. Training was terminated after 150,000,000 samples, which took between 24 and 36 hours per policy. We use the *cassie-mujoco-sim* [27] simulator, based on MuJoCo [28], to simulate Cassie during training.

Single-Gait Policy Results. We found that it was straightforward to use the probabilistic framework to train policies to learn standalone gaits, such as hopping, walking, running, or even skipping. These behaviors can be learned simply by holding the values of ratios r and cycle offset parameters $\theta_{\text{left}}, \theta_{\text{right}}$ to be constant throughout training (we refer the reader to Fig. 4 for examples of gait specifications). Videos of single-gait policies which learn skipping hopping, and walking can be found in our submission video.

Multi-Gait Policy Results By keeping the cycle offsets fixed and varying the phase ratios over some range during training, we are able to train policies that can learn to hop with more or less time in the air, and policies that can transition from walking to running. However, learning to transition between gaits by varying both the cycle offsets and phase ratios during training appears to be a challenging learning problem: policies which are trained in this fashion can end up asymmetrically walking instead of hopping, or learn other undesirable behaviors that resemble a fusion of all the different commanded gaits. To avoid these issues, we introduce additional reward terms called *transition penalties* to further distinguish each of the desired behaviors from each other during training. Transition penalties are behavior-specific costs which are only active when specific behaviors are being commanded. For hopping, which is commanded when cycle offsets are very close to each other, we activate a cost for constraining the feet positions to be close together. Similarly, by adding another transition penalty for standing, we can train a generic controller to transition between all steady state two-beat gaits and standing in place. Full details of the reward function for generic controllers are described in detail in the Appendix.

Outdoor Experiments We demonstrate the robustness of policies learned with our approach in a variety of outdoor experiments, shown in our submission video [1]. Given that the robot is effectively blind to the external world, the behaviors we observe in response to disturbances are surprisingly robust. We show the multi-gait policy hopping on and off a sidewalk, walking with one foot elevated on a curb, and running over small bumps in its path. The policy is also

[1] youtu.be/4DnxV9lko_U

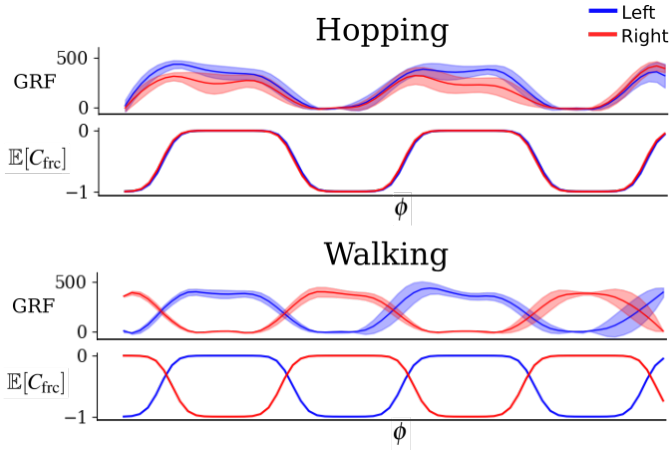


Fig. 5. Comparison between the mean measured simulation ground reaction forces (GRFs) in newtons and the corresponding expected values of C_{frc} over multiple policies. When the expectation is close to -1, the policies refrain from applying foot forces. When the expectation is close to 0, the policies apply foot forces.

shown smoothly transitioning between all of the 2-beat gaits while moving forward on a crowned road, and while turning in a small circle on a turf field. We also show a multi-gait policy descending down 5 steps of stairs while in a running gait. In the staircase and sidewalk tests, we observe the foot slipping off of small ledges and being compliant to the slope of the ground. This indicates a priority on force profile, as opposed to position only.

VI. Conclusion

In this work, we introduced a probabilistic reward framework which allows for learning of all of the common bipedal gait behaviors observed in animals. This framework requires no reference trajectories, enabling policies to explore a rich space of possible interaction with the world during training without artificial constraints to some arbitrary trajectory through space. We showed that not only are we able to train policies to learn all common bipedal gaits individually, including walking, running, hopping, galloping, and skipping, but also that we can learn all 2-beat behaviors on single policies and transition between them continuously, even while in motion. Some questions which remain unanswered include how easily this framework might be applied to the space of possible quadrupedal gaits, or other bipedal morphologies. A modified version of this framework could also be applied to aperiodic motions to learn one-off behaviors.

Appendix

Full Reward Functions for Generic 2-Beat Policies

When learning the continuum of 2-beat gaits, we find that policies often learn to stutter rather than hop with their feet together, which is not observed when training hopping standalone. To combat this, we introduce a *hop symmetry* term, a cost which becomes active when the cycle offsets of both legs match closely and punishes large distances between the feet in the sagittal and transverse planes, err_{sym},

$$q_{hop\ sym}(s) = 1 - \exp(-\text{err}_{\text{sym}} \exp(-5\left|sin(2\pi(\theta_{\text{left}} - \theta_{\text{right}}))\right|))$$

To learn a standing behavior in addition to the rest of the 2-beat gaits, the reward function must be modified to reflect our wish for the policy to remain very still when commanded. First, we define the standing region of gait parameter space as one where the swing-to-stance phase ratio is close to 0.

We define $\omega = (1 + \exp(-50(r_{\text{swing}} - 0.15)))^{-1}$, which is a coefficient close to one during normal locomotion (when the swing and stance ratios are in the range [0.35, 0.7], and close to zero during standing (where the swing phase ratio is close to zero). Now we define an additional standing cost, which applies an additional action difference cost and a foot symmetry cost (similar to the hopping symmetry) when the command inputs are in the standing region.

$$q_{\text{standing cost}}(s) = 1 - \exp(-((\text{err}_{\text{sym}}) + 20 q_{\text{action diff}}(s)))$$

Now we define the full reward for a generic policy which includes standing:

$$\begin{aligned}\mathbb{E}[R_{\text{multi}}(s,\phi)] = \quad & 0.400 \cdot \mathbb{E}[R_{\text{bipedal}}(s,\phi)] \\ + \quad & 0.300 \cdot \mathbb{E}[R_{\text{cmd}}(s)] \\ + \quad & 0.100 \cdot \mathbb{E}[R_{\text{smooth}}(s)] \\ + \quad & 0.100 \cdot (\omega - 1) \cdot q_{\text{standing cost}}(s) \\ + \quad & 0.100 \cdot (-1) \cdot q_{\text{hop sym}}(s) \\ + \quad & 1\end{aligned}$$

Quantity q_i	Detail
$q_{\text{left/right frc}}$	$1 - \exp(-\omega \lVert \text{raw_foot_frc} \rVert_2^2 / 100)$
$q_{\text{left/right spd}}$	$1 - \exp(-2 \cdot \omega \lVert \text{raw_foot_spd} \rVert_2^2)$
$q_{\dot{x}}$	$1 - \exp(-2 \cdot \omega \lvert \dot{x}_{\text{desired}} - \dot{x}_{\text{actual}} \rvert)$
$q_{\dot{y}}$	$1 - \exp(-2 \cdot \omega \lvert \dot{y}_{\text{desired}} - \dot{y}_{\text{actual}} \rvert)$
$q_{\text{orientation}}$	$1 - \exp(-3 \cdot (1 - ((\text{quat}_{\text{actual}})^T (\text{quat}_{\text{des}}))^2)$
$q_{\text{action diff}}$	$1 - \exp(-5 \cdot \lVert a_t - a_{t-1} \rVert)$
q_{torque}	$1 - \exp(-0.05 \cdot \lVert \tau \rVert)$
$q_{\text{pelvis acc}}$	$1 - \exp(-0.10 \cdot (\lVert \text{pelvis}_{\text{rot}} \rVert + \lVert \text{pelvis}_{\text{acc}} \rVert)$

TABLE II.

Where $\dot{x}_{\text{actual}}$ and $\dot{y}_{\text{actual}}$ are the current forward and lateral speeds of the pelvis. $\text{quat}_{\text{actual}}$ and quat_{des} are the actual pelvis orientation and the desired pelvis orientation, in quaternion format. a_t is the current timestep's action, and a_{t-1} is the previous timestep's action. τ is the net torque applied to all joints across the robot. $\text{pelvis}_{\text{rot}}$ is the rotational velocity of the pelvis, and $\text{pelvis}_{\text{acc}}$ is the translational acceleration of the pelvis. Note that certain terms are multiplied by ω, signifying that these terms should not be respected by the policy when $\omega \approx 0$ (i.e., the policy is somewhere inside the standing region of the gait parameter space), to prevent the policy from (for example) attempting to match a desired speed while standing. The choice of $1 - \exp(-|x|)$ as a kernel function was influenced by a desire to have a reward bounded by 0 and 1.

ACKNOWLEDGEMENTS

Thanks to Jeremy Dao, Helei Duan, and Kevin Green for stimulating conversations and help with hardware testing, and to Intel for providing access to the vLab academic compute cluster.

REFERENCES

[1] H. Zhang, S. Starke, T. Komura, and J. Saito, "Mode-adaptive neural networks for quadruped motion control," *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 1–11, 2018, ISSN: 15577368. DOI: 10.1145/3197517.3201366.

[2] S. Starke, Y. Zhao, T. Komura, and K. Zaman, "Local motion phases for learning multi-contact character movements," *ACM Transactions on Graphics*, vol. 39, no. 4, 2020, ISSN: 0730-0301. DOI: 10.1145/3386569.3392450.

[3] Z. Xie, P. Clary, J. Dao, P. Morais, J. Hurst, and M. van de Panne, "Learning locomotion skills for cassie: Iterative design and sim-to-real," L. P. Kaelbling, D. Kragic, and K. Sugiura, Eds., ser. Proceedings of Machine Learning Research, vol. 100, PMLR, 2020, pp. 317–329. [Online]. Available: http://proceedings.mlr.press/v100/xie20a.html.

[4] J. Siekmann, S. Valluri, J. Dao, L. Bermillo, H. Duan, A. Fern, and J. Hurst, "Learning Memory-Based Control for Human-Scale Bipedal Locomotion," 2020. arXiv: 2006.02402. [Online]. Available: http://arxiv.org/abs/2006.02402.

[5] X. B. Peng, P. Abbeel, S. Levine, and M. van de Panne, "DeepMimic," *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 1–14, 2018, ISSN: 0730-0301. DOI: 10.1145/3197517.3201311.

[6] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," *arXiv preprint arXiv:1606.01540*, 2016.

[7] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, 2019.

[8] Z. Gan, Y. Yesilevskiy, P. Zaytsev, and C. D. Remy, "All common bipedal gaits emerge from a single passive model," *Journal of The Royal Society Interface*, vol. 15, no. 146, p. 20 180 455, 2018.

[9] S. Levine, J. M. Wang, A. Haraux, Z. Popovic, and V. Koltun, "Continuous character control with low-dimensional embeddings," *ACM Transactions on Graphics (TOG)*, vol. 31, no. 4, pp. 1–10, 2012.

[10] A. Safonova, J. K. Hodgins, and N. S. Pollard, "Synthesizing physically realistic human motion in low-dimensional, behavior-specific spaces," *ACM Transactions on Graphics (ToG)*, vol. 23, no. 3, pp. 514–521, 2004.

[11] P.-B. Wieber and C. Chevallereau, "Online adaptation of reference trajectories for the control of walking systems," *Robotics and Autonomous Systems*, vol. 54, no. 7, pp. 559–566, 2006.

[12] D. Holden, T. Komura, and J. Saito, "Phase-functioned neural networks for character control," *ACM Transactions on Graphics*, vol. 36, no. 4, pp. 1–13, 2017, ISSN: 15577368. DOI: 10.1145/3072959.3073663.

[13] S. Starke, H. Zhang, T. Komura, and J. Saito, "Neural state machine for character-scene interactions.," 2019.

[14] S. Agrawal, S. Shen, and M. van de Panne, "Diverse motion variations for physics-based character animation," *Symposium on Computer Animation*, 2013.

[15] T. Geijtenbeek, N. Pronost, A. Egges, and M. H. Overmars, "Interactive character animation using simulated physics.."

[16] K. Bergamin, S. Clavet, D. Holden, and J. R. Forbes, "Drecon: Data-driven responsive control of physics-based characters," *ACM Transactions on Graphics (TOG)*, vol. 38, no. 6, pp. 1–11, 2019.

[17] X. B. Peng, G. Berseth, and M. Van de Panne, "Terrain-adaptive locomotion skills using deep reinforcement learning," *ACM Transactions on Graphics (TOG)*, vol. 35, no. 4, pp. 1–12, 2016.

[18] Y. Tassa, Y. Doron, A. Muldal, T. Erez, Y. Li, D. de Las Casas, D. Budden, A. Abdolmaleki, J. Merel, A. Lefrancq, T. P. Lillicrap, and M. A. Riedmiller, "Deepmind control suite," *CoRR*, vol. abs/1801.00690, 2018. arXiv: 1801.00690. [Online]. Available: http://arxiv.org/abs/1801.00690.

[19] T. Haarnoja, S. Ha, A. Zhou, J. Tan, G. Tucker, and S. Levine, "Learning to walk via deep reinforcement learning," *arXiv preprint arXiv:1812.11103*, 2018.

[20] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-real: Learning agile locomotion for quadruped robots," in *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, Jun. 2018. DOI: 10.15607/RSS.2018.XIV.010.

[21] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[22] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[23] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*.

[24] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-real: Learning agile locomotion for quadruped robots."

[25] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal Policy Optimization Algorithms*, 2017. arXiv: 1707.06347 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1707.06347.

[26] F. Adbolhosseini, H. Y. Ling, Z. Xie, X. B. Peng, and M. van de Panne, "On learning symmetric locomotion," in *Proc. ACM SIGGRAPH Motion, Interaction, and Games (MIG 2019)*, 2019.

[27] Agility Robotics, *Cassie-mujoco-sim*, 2018. [Online]. Available: https://github.com/osudrl/cassie-mujoco-sim.

[28] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE, 2012, pp. 5026–5033.